

Understanding Basic Commands using OpenSTA

INTRODUCTION:

In this lab, students will learn the basic of OpenSTA in a static timing analysis (STA) tool used to verify the timing correctness of digital circuits at the gate level. It analyses the timing paths in a design based on the netlist, timing libraries, and design constraints to check for violations of setup and hold times. It calculates delays caused by cells and interconnects and identifies critical paths that affect the maximum operating frequency. It is widely used in open-source EDA flows to ensure designs meet timing requirements before manufacturing. By providing detailed timing reports, it helps designers optimize and fix timing issues to guarantee reliable chip operation.

OBJECTIVE:

After completing this lab, you will be able to:

- Load Liberty timing libraries into OpenSTA
- Load synthesized gate-level netlist for STA
- Apply SDC constraints (clocks, input/output delays)
- Perform timing propagation and analyse paths
- Report setup/hold timing violations
- Understand critical path reports and slack values

COMMANDS:

The command line arguments for STA are shown below.

Commands	Description
sta	To invoke the tool
-help	- Show help and exit
-version	- Show version and exit
-no_init	- Do not read ~/.sta
-no_splash	- Do not print the splash message
-threads count max	- Use count threads
-exit	- Exit after reading cmd_file
cmd_file	- Source cmd_file

Note:

- If a TCL command file cmd_file is specified on the command line, commands are read from the file and executed before entering an interactive TCL command interpreter.
- The –threads option specifies how many parallel threads to use. Use –threads max to use one thread per processor.

Commands	Description
read_liberty osu018_stdcells.lib	To read a design into OpenSTA use the read_liberty command to read Liberty library files.
read_verilog <design_synth.v>	Read hierarchical structural Verilog files with the read_verilog command.
link_design <design_top_module>	The link_design command links the Verilog to the Liberty timing cells.
read_sdc <design.sdc>	Timing constraints can be entered as TCL commands or read using the read_sdc command.
report_checks	For checking the reports

Note: Any number of Liberty and Verilog files can be read before linking the design.

➤ **For Timing Analysis with Multiple Process Corners:**

```

define_corners wc typ bc //to analysis corners

read_liberty -corner wc example1_slow.lib //loads data from a liberty format
library file.

read_liberty -corner typ example1_typ.lib // loads data from a liberty
format library file.

read_liberty -corner bc example1_fast.lib // loads data from a liberty
format library file.

read_verilog example1.v // to read verilog code

link_design top // link the verilog netlist to libraries

set_timing_derate -early 0.9 //timing derate factor to early (min) delay path
    
```

```
set_timing_derate -late 1.1 //timing derate factor to early (max) delay path

create_clock -name clk -period 10 {clk1 clk2 clk3} // clock constraints

set_input_delay -clock clk 0 {in1 in2} // input constraints

report_checks // used to generate timing reports
```

➤ Power Analysis:

```
read_liberty sky130hd_tt.lib // loads data from a liberty format
                                library file.

read_verilog gcd_sky130hd.v // to read verilog code

link_design gcd //link the verilog netlist to libraries

read_sdc gcd_sky130hd.sdc // read the sdc which contain timing constraints

report_power // OpenSTA supports static power analysis with the report_power command
```

```
➔ report_annotated_check [-setup] // report annotated setup checks
                        [-hold] // report annotated hold checks
                        [-recovery]// report annotated recovery checks
                        [-removal]// report annotated removal checks
                        [-nochange]//report annotated nochange checks
                        [-width] // report annotated width checks
                        [-period]//report annotated period checks
                        [-max_skew]//report annotated max skew check
                        [-max_line lines]//maximum number of
                                lines listed by -list_annotate and
                                -list_not_annotated options
                        [-list_annotated]// list annotated timing
                                arcs
                        [-list_not_annotated]// list unannotated
                                timing arcs
                        [-constant_arcs]
```

➔ **report_annotated_delay [-cell]**

[-net]

[-from_in_ports]

[-to_out_ports]

[-max_lines lines]

[-list_annotated]

[-list_not_annotated]

[-constant_arcs]

-cell //Report annotated cell delays.

-net //Report annotated internal net delays.

-from_in_ports //Report annotated delays from input ports.

-to_out_ports //Report annotated delays to output ports.

-max_lines //lines Maximum number of lines listed by the **-list_annotated** and **-list_not_annotated** options.

-list_annotated //List annotated timing arcs.

-list_not_annotated //List unannotated timing arcs.

-constant_arcs //Report separate annotation counts for arcs

➔ **report_cell [-connections]**

[-verbose]

instance_path

[> filename]

[>> filename]

-connections // Report the instance pins.

-verbose //With **-connections** also report all pins connected to each instance pin net.

instance_path //Hierarchical path to the instance.

➔ **Report_checks [-path_delay min]**

[-path_delay min_rise]

[-path_delay min_fall]

```
[-path_delay max]
[-path_delay max_rise]
[-path_delay max_fall]
[-path_delay min_max]
-corner corner
-slack_min min_slack
-slack_max max_slack
-sort_by_slack
```

```
[-path_delay min] //Report min path (hold) checks
[-path_delay min_rise] //Report min path (hold) checks for rising endpoints
[-path_delay min_fall] //Report min path (hold) checks for falling
                        endpoints
[-path_delay max] //Report max path (setup) checks
[-path_delay max_rise] //Report max path (setup) checks for rising
                        endpoints
[-path_delay max_fall] //Report max path (setup) checks for falling
                        endpoints
[-path_delay min_max] //Report max and max path (setup and hold) checks
-corner corner
-slack_min min_slack //Only report paths with less slack than min_slack
-slack_max max_slack //Only report paths with less slack than max_slack
-sort_by_slack //Sort paths by slack rather than slack grouped by path
                group
```

```
➔ report_check_types [-violators]
                    [-verbose]
                    [-format slack_only|end]
                    [-max_delay]
                    [-min_delay]
                    [-recovery]
                    [-removal]
                    [-clock_gating_setup]
```

```
[-clock_gating_hold]
[-max_slew]
[-max_skew]
[-min_slew]
[-min_pulse_width]
[-min_period]

-violators //Report all violated timing and design rule constraints
-verbose //Use a verbose output format
-format slack_only //Report the minimum slack for each timing check.
-format end //Report the endpoint for each check
-max_delay //Report setup and max delay path delay constraints
-min_delay //Report hold and min delay path delay constraints
-recovery //Report asynchronous recovery checks
-removal //Report asynchronous removal checks
-clock_gating_setup //Report gated clock enable setup checks
-clock_gating_hold //Report gated clock hold setup checks
-max_slew //Report max transition design rule checks
-max_skew //Report max skew design rule checks
-min_pulse_width //Report min pulse width design rule checks
-min_period //Report min period design rule checks
-min_slew //Report min slew design rule checks

➔ report_clock_skew [-setup|-hold]
                [-clock clocks]
                [-digits digits]
    -setup //Report skew for setup checks
    -hold //Report skew for hold checks
    -clock clocks //The target clocks to report
    -digits digits //The number of digits to report for delays

➔ report_instance [-connections]
                [-verbose]
                instance_path
```

[> filename]

[>> filename]

-connections //Report the pins connected to the net

-verbose //Report the connections of each pin

instance_path //Hierarchical path to a instance

➔ **report_power** [-instances instances]

[-digits digits]

[> filename]

[>> filename]

-instances instances //Report the power for each instance of instances. If the instance is hierarchical the total power for the instances inside the hierarchical instance is reported.

-digits digits //The number of digits after the decimal point to report. The default value is the variable sta_report_default_digits.

➔ **Report_pluse_width_checks** [-verbose]

[-digits digits]

[-no_line_splits]

[pins]

[> filename]

[>> filename]

-verbose //Use a verbose output format

-digits digits //The number of digits after the decimal point to report. The default value is the variable sta_report_default_digits.

-no_line_splits

pins //List of pins or ports to report

➔ **report_units**

➔ **report_worst_slack** [-min]

[-max]

[-digits digits]

-max //Report the worst max/setup slack

-min //Report the worst min/hold slack

-digits digits //The number of digits after the decimal point to report. The default value is the variable sta_report_default_digits

➔ **set_clock_gating_check** [-setup setup_time]
[-hold hold_time]
[-rise]
[-fall]
[-high]
[-low]
[objects]

-setup setup_time //Clock enable setup margin

-hold hold_time //Clock enable hold margin

-rise //The setup/hold margin is for the rising edge of the clock enable

-fall //The setup/hold margin is for the falling edge of the clock enable

-high //The gating clock is active high (pin and instance objects only)

-low //The gating clock is active low (pin and instance objects only).

objects //A list of clocks, instances, pins or ports.

➔ **Set_clock_latency** [-source]
[-clock clock]
[-rise]
[-fall]

[-min]

[-max]

delay

objects

-source //The latency is at the clock source.

-clock *clock* //If multiple clocks are defined at a pin this use this option to specify the latency for a specific clock.

-rise //The latency is for the rising edge of the clock.

-fall //The latency is for the falling edge of the clock.

-min //delay is the minimum latency.

-max //delay is the maximum latency.

delay //Clock source or insertion delay.

objects //A list of clocks, pins or ports.

➔ **set_clock_transition [-rise]**
[-fall]
[-min]
[-max]
transition
clocks

-rise //Set the transition time for the rising edge of the clock.

-fall //Set the transition time for the falling edge of the clock.

-min //Set the min transition time.

-max //Set the min transition time.

transition //Clock transition time (slew).

clocks //A list of clocks.

➔ **set_hierarchy_separator *separator***

separator // Character used to separate hierarchical names.

```

→ set_ideal_latency [-rise]
                        [-fall]
                        [-min]
                        [-max]
                        delay
                        objects
    
```

The set_ideal_latency command is parsed but ignored.

```

→ set_max_delay [-rise]
                  [-fall]
                  [-from from_list]
                  [-rise_from from_list]
                  [-fall_from from_list]
                  [-through through_list]
                  [-rise_through through_list]
                  [-fall_through through_list]
                  [-to to_list]
                  [-rise_to to_list]
                  [-fall_to to_list]
                  [-reset_path]
                  [-ignore_clock_latency]
                  delay
    
```

-rise //Set max delay for rising paths.

-fall //Set max delay for falling paths.

-from from_list //A list of clocks, instances, ports or pins.

-through through_list //A list of instances, pins or nets.

-to to_list //A list of clocks, instances, ports or pins.

-ignore_clock_latency //Ignore clock latency at the source and target registers.

-reset_path //Remove any matching set_false_path, set_multicycle_path, set_max_delay, set_min_delay exceptions first.

delay //The maximum delay

➔ **set_max_transition** [-data_path]
[-clock_path]
[-rise]
[-fall]
transition
objects

-data_path //Set the max slew for data paths.

-clock_path //Set the max slew for clock paths.

-rise //Set the max slew for rising paths.

-fall //Set the max slew for falling paths.

transition //The maximum slew/transition time.

objects //List of clocks, ports or designs.

➔ **set_output_delay** [-rise]
[-fall]
[-max]
[-min]
[-clock *clock*]
[-clock_fall]
[-reference_pin *ref_pin*]
[-source_latency_included]
[-network_latency_included]
[-add_delay]
delay
port_pin_list

-rise //Set the output delay for the rising edge of the input.

-fall //Set the output delay for the falling edge of the input.

-max //Set the maximum output delay.

-min //Set the minimum output delay.

-clock *clock* //The external check is to clock. The default clock edge is rising.

-clock_fall //The external check is to the falling edge of clock.

-reference_pin ref_pin //The external check is clocked by the clock that arrives *ref_pin*.

-add_delay //Add this output delay to any existing output delays.

delay //The external delay to the check clocked by clock.

pin_port_list //A list of pins or ports.

➔ **set_power_activity [-global]**

[-input]

[-input_ports ports]

[-pins pins]

[-activity activity]

[-duty duty]

-global //Set the activity/duty for all non-clock pins.

-input //Set the default input port activity/duty.

-input_ports ports //Set the input port activity/duty.

-pins pins //Set the pin activity/duty.

-activity activity //The activity, or number of transitions per clock cycle.

-duty duty //The duty, or probability the signal is high. Defaults to 0.5.

CONCLUSION:

The OpenSTA lab demonstrates the complete static timing analysis flow on a simple gate-level design, covering key commands from reading Liberty libraries, Verilog netlists, and SDC constraints to linking and analysing the design. Each command plays a crucial role in accurately propagating timing, checking paths, and reporting violations like setup and hold time failures. The generated timing reports confirm if the design meets timing requirements, providing critical insights for designers to optimize timing closure. Mastering OpenSTA commands offers a robust foundation to handle timing analysis in complex VLSI designs effectively.